Let's think about timing for a moment. Unlike our redis and database services, which have no dependencies, our web service relies on the database service being available in order to start successfully (it also relies on redis, but as we only connect in a controller, this wouldn't cause an error at startup). However, the *order* that services launch is indeterminate; it is affected by image size, download speeds, load, and, importantly, how long it takes for the container's command to have performed any initialization. What happens if our web service launches before the database service is ready for connections?

We have largely been shielded from having to care about this lack of determinism because—thanks to Swarm's default behavior—our web service automatically restarts if it fails to launch for any reason, including being unable to connect to the database. However, using a one-shot container changes things. If our planned database-migrator service launched before Postgres was ready for connections, it would fail with a PG::ConnectionBad error, and terminate. Our one-shot container would have missed its one chance, leaving our database unmigrated. Not a pretty sight.

Generally, we should design our apps to be resilient to services coming and going, and not being available when expected.[7] Here, a good solution is to wrap the rails db:migrate command with a script that waits until the database is not just started, but ready to handle connections. You can think of this as a *readiness health check*. We're going to use a shell script called wait-for[8] to perform this checking for us.

Download the wait-for script in your Rails root folder.

We'll need to include this file in our myapp_web:prod image, so we'll rebuild the image shortly. However, the wait-for script relies on having the netcat Unix tool[9] available in order to determine whether the database is accepting connections on its designated port, so we'll need to install that too.

Modify your Dockerfile.prod file to add netcat to the apt-get install command:

```
RUN apt-get update -yqq && apt-get install -yqq --no-install-recommends \
  netcat \ # needed for `wait-for` TCP connection checking
  nodejs \
  yarn
```

We also need to make the wait-for script executable inside the image. On Linux or macOS, you can do this by running chmod +x wait-for locally; the file will

7. https://docs.docker.com/compose/startup-order/
8. https://github.com/mrako/wait-for
9. https://en.wikipedia.org/wiki/Netcat